RingGSW based Universally Composable Oblivious Transfer

Group No. 22

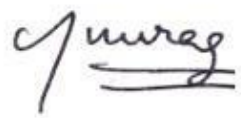
Guide: Dr. Kunwar Singh

Members:

- Anurag Goyal – 106119014
- Manmohan Prajapat – 106119072
- Sobhagya Singh Dewal - 106119117



Dr. Kunwar Singh



Anurag Goyal



Manmohan Prajapat



Sobhagya Singh Dewal

Introduction

We have constructed a secure method for transferring a message without knowing which one the receiver wants. The receiver also can’t know the other messages. This is called 1-out-of-n Oblivious Transfer (OT) protocol.

We use a fully homomorphic encryption (FHE) scheme. It allows us to perform operations on ciphertexts in an untrusted environment to achieve the same results on the resulting plaintext.

The protocol is universally composable. Its post-quantum security comes from the RLWE hard problem. We achieved this by converting Craig Gentry’s FHE scheme (GSW) to its ring variant (RGSW).

Problem Statement and Objectives

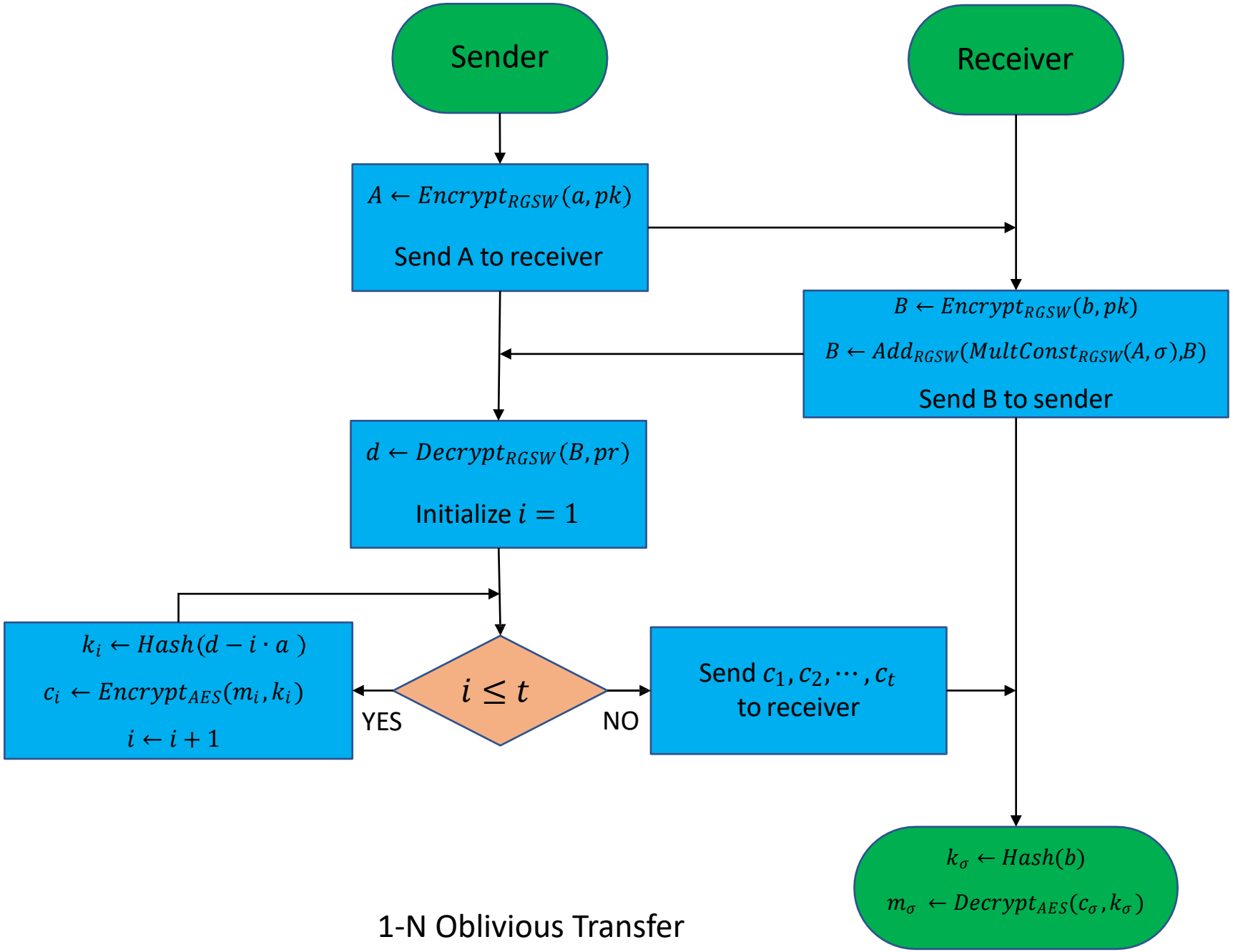
Problem Statement

- There is a need for a secure and reliable OT with the capacity to resist attacks from quantum algorithms.
- Conventional lattice-based HE schemes don't provide full homomorphism because of uncontrolled noise growth with each operation.
- Additionally, the existing LWE based FHE schemes suffer from huge public key and ciphertext sizes. Conversion to ring variants improve the protocol's efficiency without compromising its security.

Objectives

- To implement a lattice-based asymmetric cryptosystem with full homomorphism to enable secure computation without the knowledge of the plaintext.
- To construct a ring variant of the FHE scheme based on the RLWE assumption.
- To ensure the OT protocol is universally composable and post-quantum secure under the RLWE assumption.

Block Diagram



1-N Oblivious Transfer

Algorithm

Polynomial ring $R = Z[x]/\Phi_{2n}(x)$ where $\Phi_{2n}(x)$ is a cyclotomic polynomial defined as $\Phi_{2n}(x) = x^n + 1$ with n a power of 2. Define $R_q = R/qR$, i.e., all coefficients of a polynomial in R_q are modulo q.

The coefficient modulus is $q = 2^k$ and the polynomial ring $R_q \leftarrow Z_q[x]/\Phi_{2n}(x)$ for $n = 2^{k'}$. Define $N = 2k$.

RGSW variant functions of the GSW scheme are defined for a polynomial $a \in R_q$ with coefficients $a_0, a_1, \cdots, a_{n-1} \in Z_q$

$$\overrightarrow{a'} = BitDecomp_{RGSW}(a) = \left(\sum_{i=0}^{n-1} a_{i,0} \cdot x^i, \cdots, \sum_{i=0}^{n-1} a_{i,k-1} \cdot x^i \right) \in R_q^k$$

$a_{i,j} \in Z_q$ denotes the jth LSB in a_i, therefore, $a_{i,j} \in \{0, 1\}$ for $0 \leq i < n, 0 \leq j < k$

$$BitDecomp_{RGSW}^{-1}(\overrightarrow{a'}) = \sum_{i=0}^{n-1} \left(\sum_{j=0}^{k-1} 2^j \cdot a_{i,j} \right) \cdot x^i \in R_q$$

Here, $a_{i,j} \in Z_q$ need not be only 0 or 1.

$$Flatten_{RGSW}(\vec{a}) = BitDecomp_{RGSW}\left(BitDecomp_{RGSW}^{-1}(\vec{a})\right) \in R_q^k$$
$$Powersof2_{RGSW}(a) = \left(a, 2a, \cdots, 2^{k-1}a\right) \in R_q^k$$

For a matrix $A \in R_q^{m \times n}$, all the functions above operate on each row of A.

RingGSW FHE Scheme

Setup

Let $\chi_{\mu,\sigma} \leftarrow$ Gaussian Distribution and $I_N \in R_q^{N\times N}$ is the identity matrix of order N.

Secret Key Generation: $t, \vec{v} \leftarrow SecretKeyGen_{RGSW}(q, n)$

Choose $t \in R_q$ at random, then $sk \leftarrow \vec{s} = (1, -t) \in R_q^2$. The secret key is $\vec{v} = Powersof2_{RGSW}(\vec{s}) = \left(1, 2, \cdots, 2^{k-1}, -t, -2t, \cdots, -2^{k-1}t\right) \in R_q^N$

Public Key Generation: $A \leftarrow PublicKeyGen_{RGSW}(t)$

Choose $b_1 \in R_q$ and $e \leftarrow \chi \in R_q$ is an error polynomial with small coefficients. Define $b_2 \leftarrow b_1 \cdot t + e$ then the public key is the matrix $A = [b_2 \quad b_1] \in R_q^{1\times 2}$. Observe that $A \cdot \vec{s} = \vec{e}$

Encryption: $C \leftarrow Encrypt_{RGSW}(z, A)$

Choose a plaintext $z \in R_q$ and a uniform $\vec{r} \in R_q^N$ such that the coefficients of any member polynomial are from $\{0, 1\}$. In other words, $\vec{r} \leftarrow \left(\sum_{i=0}^{n-1} y_{0,i} \cdot x^i, \cdots, \sum_{i=0}^{n-1} y_{N-1,i} \cdot x^i\right) \in R_q^N$ where $y_{i,j} \in \{0, 1\} \subset Z_q$ for $0 \le i < N, 0 \le j < k$

The ciphertext is given by $C = Flatten_{RGSW}\left(z \cdot I_N + BitDecomp_{RGSW}(\vec{r} \cdot A)\right) \in R_q^{N\times N}$

Decryption: $m \leftarrow Decrypt_{RGSW}(C, \vec{v})$

Let $z_0, z_1, \cdots, z_{k-1} \in Z_q$ be such that $\sum_{j=0}^{n-1} z_j \cdot x^j = z$ in R_q.

Consider $\vec{b} = (b_1, b_2, \cdots, b_{k-1}) = \left(z + e_1, 2z + e_2, \cdots, 2^{k-1}z + e_{k-1}\right)$.
Initialize $z \leftarrow 0 \in R_q$

for $j = 0, 1, \cdots, n-1$:

$z_j \leftarrow 0 \in Z_q$

for $i = k-1, \cdots, 1, 0$:

$$z_j \leftarrow z_j + 2^{k-1-i} \cdot \langle b_{i,j} - 2^i z_j \notin \left[-\frac{q}{4}, \frac{q}{4}\right) mod\ q \rangle\ mod\ q$$

$z \leftarrow z + z_j \cdot x^j$

Where $b_{i,j} \in Z_q$ is the coefficient of x^j in b_i and $\langle E \rangle = \begin{cases} 1, & E\ is\ true \\ 0, & E\ is\ false \end{cases}$ The $z \in R_q$ obtained is the decrypted plaintext.

Homomorphic Operations

Constant Multiplication: $C' \leftarrow MultConst_{RGSW}(C, \alpha)$

$C' = Flatten_{RGSW}(Flatten_{RGSW}(\alpha \cdot I_N) \cdot C) \in R_q^{N \times N}$ The error increases by a factor of at most N.

Addition: $C' \leftarrow Add_{RGSW}(C_1, C_2)$

$C' = Flatten_{RGSW}(C_1 + C_2) \in R_q^{N \times N}$ The correctness of this operation is immediate.



Implementation

The RGSW cryptosystem is implemented using SageMath, a free open-source mathematics software programming language.

The OT protocol consists of two entities, sender and receiver. These are implemented as parties in a server-client model that communicate with each other over a network.

SageMath 9.3 Shell

```
(sage-sh) meet1@LAPTOP-R2K8452O:implementation$ sage server.py

[*] listening on port 3000

connection accepted

messages (visible to sender):

0. 551c6380eb8bd4dfc5cec934775805ee
1. 6cae9bdf78c4bbf34704f19e14cdfd5f
2. ecca5b93d5497840a306f7345b3b8a7b
3. fc2f74a598b062ab6a72e0ab24281dbe
4. 31ec32d9e3e379b7013057e43965ce06
5. 3ebf9dd2c35619be0d8b6b633f7f1d09

sharing parameters


sending a random encryption (A)

recieved homomorphic encryption of A (B)

sending symmetric AES ciphertexts

[*] connection closed

(sage-sh) meet1@LAPTOP-R2K8452O:implementation$
```

SageMath 9.3 Shell

```
(sage-sh) meet1@LAPTOP-R2K8452O:implementation$ sage client.py

connected to sender

received parameters (ring, public key)

enter your choice(s) (0 to 5): 3

recevied the random encryption (A)

sending homomorphic encryption of A (B)

recevied symmetric AES ciphertexts:

0. af3533b0bb959d573932fde3ee7a9d0ddba128c9b97cbbf7e1
d67e100feac0e62f54790d40f0373f8f5905bd288400fb
1. d4fa2f13c02d8623e85c430046b6e0699e9ba7991d1a86b2d4
d0d9b8878e35cae7bbcd299ab6dee228054285ceaa8743
2. eb27f44182afc7580d9bfd4bf6717e3fa20fbde35a1a25d1b9
5e426d614961aabcc26759622a4174216cf1845b5668aa
3. 8e8ca353f2c560a7ab159fa62689bb39c1848ab23844853d1a
54527670508f57db40cb25d5b2cf7bdae3dd2f95eb57b8
4. a8254f5626da00af25b67e1d9d1e22714c68e9a7e0e2a05c79
0808d652e415cb23f58e2414d0b503f7c4000c22228e24
5. fedd5eec827bb6c4561c95fcd18f80b14d71e6245622114549
e25cf3e4cc04bbe09d5f805751485ff05e8b17652fa9ac

decrypted message:
3. fc2f74a598b062ab6a72e0ab24281dbe


[*] connection closed

(sage-sh) meet1@LAPTOP-R2K8452O:implementation$
```

1-N Oblivious Transfer

SageMath 9.3 Shell

```
(sage-sh) meet1@LAPTOP-R2K8452O2:implementation$ sage server.py

[*] listening on port 3000

connection accepted

messages (visible to sender):

0. c8a10f35546ae7ea9d7b2f5c9a0d23d7
1. 952a04c8540c7e41000fa995b0387c81
2. 08ddf2f0dd95177ba0a6fa67722ec466
3. dd5be20570cc64f93514e9cc6d3701e7
4. 92779ae0c240a465adeb7deec8d23dca
5. e50daad17a3a8ed73901a7d821795055

sharing parameters

sending a random encryption (A, T_A)

received homomorphic encryptions of A (U, T_B)

sending dummy secret key

sending symmetric AES ciphertexts

[*] connection closed

(sage-sh) meet1@LAPTOP-R2K8452O2:implementation$
```

SageMath 9.3 Shell

```
(sage-sh) meet1@LAPTOP-R2K8452O2:implementation$ sage client.py

connected to sender

received parameters (ring, public key)

enter your choice(s) (0 to 5): 0, 3, 4

received the random encryption (A, T_A)

sending homomorphic encryptions of A (U, T_B)

received dummy secret key
verification: [OK]

received symmetric AES ciphertexts:
0. 555c5819f9a5288403c2266fdb716f48e3adb4bace3f5170a9c87b63594da11136cf47367d
1ad8e5ad6f28a7a10d0fc1
1. f5d968f125af2695c8388f072f734ba214a6e9478b278fccf41cf480a6831fa523f378e683
3fc16d5321d1374769ec54
2. 7921f681ec50cfce176afaa5e14a5ceefe8ec380c2d6a77cae788c5902445bbc35a9b11a21
5cc4fa2b016c32fc93b3e8
3. 475ad648df65d91f4e628459b83883fa0983769e444da63d7b34429306af956e2caeb313c2
3b3cf1031e4c63f838b9e4
4. fbeb17cad718759e33a1179ac56f36e8bfe8fe852522be11b52f0043ad580ee5897b04e13f
f6d84284545f6de131a538
5. 67807298426d33a3010c41155b3d4def7a99ea7c6606171b577b128f89f1a00427a9224802
ea369a84105578da6dae37

decrypted message(s):

0. c8a10f35546ae7ea9d7b2f5c9a0d23d7
3. dd5be20570cc64f93514e9cc6d3701e7
4. 92779ae0c240a465adeb7deec8d23dca

[*] connection closed

(sage-sh) meet1@LAPTOP-R2K8452O2:implementation$
```

K-N Oblivious Transfer

References

[1] Esmaeilzade, Saeid & Pakniat, Nasrollah & Eslami, Ziba. (2022). A generic construction to build simple oblivious transfer protocols from homomorphic encryption schemes. The Journal of Supercomputing. 78. 10.1007/s11227-021-03826-0.

[2] Canetti, R.. (2001). Universally Composable Security: A New Paradigm for Cryptographic Protocols. IACR Cryptology ePrint Archive. 2000. 136 - 145. 10.1109/SFCS.2001.959888.

[3] Regev, Oded. (2005). On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. Journal of the ACM (JACM). 56. 84-93. 10.1145/1568318.1568324.

[4] Peikert, Chris & Pepin, Zachary. (2019). Algebraically Structured LWE, Revisited. 10.1007/978-3-030-36030-6_1.

[5] Gentry, C., Sahai, A., Waters, B. (2013). Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In: Canetti, R., Garay, J.A. (eds) Advances in Cryptology – CRYPTO 2013. CRYPTO 2013. Lecture Notes in Computer Science, vol 8042. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-40041-4_5

[6] Micciancio, D., Peikert, C. (2012). Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller. In: Pointcheval, D., Johansson, T. (eds) Advances in Cryptology – EUROCRYPT 2012. EUROCRYPT 2012. Lecture Notes in Computer Science, vol 7237. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-29011-4_41

[7] Chen, H., Dai, W., Kim, M., Song, Y. (2021). Efficient Homomorphic Conversion Between (Ring) LWE Ciphertexts. In: Sako, K., Tippenhauer, N.O. (eds) Applied Cryptography and Network Security. ACNS 2021. Lecture Notes in Computer Science, vol 12726. Springer, Cham. https://doi.org/10.1007/978-3-030-78372-3_18